

Reviewer Pack — Max-Influence DPLL Branching Matches KKL Bound on Random 3-C...

Entry 211ba56d901e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 18:46:15 UTC

Max-Influence DPLL Branching Matches KKL Bound on Random 3-CNF

Entry ID: 211ba56d901e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 18:46:15 UTC

1. Conjecture

Field A (mathematical branch): KKL Fourier-influence theory (Kahn-Kalai-Linial; Friedgut influence/junta toolkit) **Field B** (complexity object): DPLL leaf count on random 3-CNF with influence-greedy branching

Statement:

Let F be a 3-CNF on n variables at clause density $\alpha \in [3.0, 4.2]$, let $f_F: \{0,1\}^n \rightarrow \{0,1\}$ be its satisfying-assignment indicator, and let $I_{\max}(F) = \max_{i \in [n]} \Pr_{x \sim U}[f_F(x) \neq f_F(x \oplus e_i)]$ be its maximum single-variable influence. Let $\tau_{\text{KKL}}(F)$ be the leaf count of a DPLL run that uses unit-propagation and, at every internal node with residual formula $F|_\alpha$ and residual indicator $f\{F|_\alpha\}$, branches on the unassigned variable x_i maximizing $\inf_i(f\{F|_\alpha\})$ (recomputed per node). Conjecture: for every random 3-CNF F at the densities above and every $n \leq 40$, $\log_2(\tau_{\text{KKL}}(F)+1) \leq (1 - I_{\max}(F)) \cdot (n - 1) + 5\sqrt{n}$; a single instance violating this strict inequality falsifies the conjecture.

Rationale (proposer's reasoning):

KKL guarantees that any non-trivially-balanced Boolean function has a variable of influence $\Omega(\log n / n)$, and Friedgut's junta theorem says low-total-influence functions concentrate on few coordinates — together these suggest influence-greedy branching repeatedly cleaves the satisfying set along its 'pivot axis'. Testing whether the KKL-advantage compounds through restrictions probes whether influence is preserved (or amplified) under conditioning, a question untouched in the SAT-solver literature. The bound is an upper bound on an explicit deterministic algorithm's leaf count on a specific input distribution, so it is unrelated to lower bounds against P/poly and immune to the Razborov-Rudich natural-proofs trap.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cd303d04d9f751f5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 240 trials ($n \in \{8, 10, 12, 14\} \times \alpha \in \{3.5, 4.0\} \times 30$ seeds 0-29), compute $\Delta = (1 - I_{\max})(n-1) + 5\sqrt{n} - \log_2(\tau_{\text{KKL}} + 1)$. Conjecture is SUPPORTED iff $\Delta \geq 0$ on every trial (support_fraction = 1.0); a single trial with $\Delta < 0$ FALSIFIES.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.99	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.99	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - DPLL branching heuristic Fourier influence random 3-SAT - KKL influence Boolean function satisfiability backtracking complexity - maximum influence variable branching random 3-CNF leaf count junta

Top relevant hits considered: - [<http://arxiv.org/abs/1402.4455v1>] Symbiosis of Search and Heuristics for Random 3-SAT - [<http://arxiv.org/abs/cs/0506069v1>] A generating function method for the average-case analysis of DPLL - [<http://arxiv.org/abs/1908.05361v2>] Placing quantified variants of 3-SAT and Not-All-Equal 3-SAT in the polynomial hierarchy - [<http://arxiv.org/abs/1103.5740v2>] Generating and Searching Families of FFT Algorithms - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/1804.02478v5>] Complexity of the CNF-satisfiability problem - [<http://arxiv.org/abs/2410.15822v2>] Learning junta distributions, quantum junta states, and QAC⁰ circuits - [<http://arxiv.org/abs/1510.05486v2>] Two types of branching programs with bounded repetition that cannot efficiently compute monotone 3-CNFs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.7s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_3cnf(n, alpha):
    clauses = []
    for _ in range(int(alpha * n)):
        clause = [random.choice([-1, 1]) * (i + 1) for i in random.sample(range(n), 3)]
        clauses.append(clause)
    return clauses

def truth_table(f, n):
    table = {}
    for i in range(2**n):
```

```

        assignment = [(i >> j) & 1 for j in range(n)]
        table[tuple(assignment)] = f(assignment)
    return table

def influence(f, n):
    inf = [0] * n
    for var in range(n):
        flips = {tuple(assignment[:var] + [1-assignment[var]] + assignment[var+1:]) for assignment in truth_table(f, n)}
        inf[var] = sum(truth_table(f, n)[flip] != f(list(flip)) for flip in flips) / (2**n)
    return inf

def dpll(alpha, assignment):
    if not any clause for clause in alpha:
        return 1
    var = min(range(len(assignment)), key=lambda v: sum(1 for c in alpha if any(assignment[v] != lit for lit in enumerate(c))))
    left_assignment = assignment[:var]
    right_assignment = assignment[var:]
    left_assignment[var] = 0
    right_assignment[var] = 1
    return dpll(alpha_new(left_assignment), left_assignment) + dpll(alpha_new(right_assignment), right_assignment)

def alpha_new(assignment):
    return [c for c in alpha if any(assignment[var] != lit for var, lit in enumerate(c))]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([8, 10, 12, 14])
    alpha = generate_3cnf(n, random.uniform(3.5, 4.0))
    f_F = lambda x: all(c for c in alpha if any(x[var] != lit for var, lit in enumerate(c)))
    table = truth_table(f_F, n)
    inf = influence(f_F, n)
    I_max = max(inf)
    tau_KKL = dpll(alpha, [0] * n)
    metric_value = math.log2(tau_KKL + 1)
    conjecture_holds = metric_value <= (1 - I_max) * (n - 1) + 5 * math.sqrt(n)
    counterexample = "" if conjecture_holds else f"tau_KKL={tau_KKL}, I_max={I_max}"
    return {
        "metric_name": "log2(tau_KKL + 1)",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}, \"instances_tested\": {result['instances_tested']}, \"conjecture_holds\": {result['conjecture_holds']}, \"counterexample\": \"{result['counterexample']}\"}}")
        results.append(result)

    mean_metric = sum(r["metric_value"] for r in results) / len(results)
    std_metric = math.sqrt(sum((r["metric_value"] - mean_metric)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con
    print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)]['cour
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41947201.py", line 76, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41947201.py", line 58, in run_trial
    inf = influence(f_F, n)
           ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41947201.py", line 35, in influence
    flips = {tuple(assignment[:var] + [1-assignment[var]] + assignment[var+1:]) for assignment in truth
              ~~~~~^~~~~~
TypeError: can only concatenate tuple (not "list") to tuple

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in the `influence()` function (tuple/list concatenation bug) before any trials completed, so no data was produced to evaluate the pre-registered $\Delta \geq 0$ criterion across the 240 trials. | next: Fix the `influence()` helper by converting the assignment tuple to a list (or building the flipped tuple via slicing on tuples) and rerun the full 240-trial sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	160970
2	preregistration	claude_max	opus	0	0	6022
3	novelty	claude_max	opus	0	0	4880
4	novelty	claude_max	opus	0	0	8581

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15051
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14017
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15049
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12361
9	judge	claude_max	opus	0	0	4157

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 241088 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in research/audit/211ba56d901e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/211ba56d901e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/211ba56d901e.tar.gz (if generated)